

Secure Heap: Heap implementation with Canaries

Måns Abrahamsson, Emil Göransson,
Lukas Lannge, Samuel Sendek

Fall 2025

1 Heap Buffer Overflow

A process operates within a dedicated memory space containing the instructions and data necessary for execution. This space is divided into several distinct segments: the Code segment (machine instructions), the Data segment (global variables), the Stack (local variables), and the Heap.

Unlike other segments defined at compile time, the Heap is designed for dynamic memory allocation. It allows data structures to grow or shrink as needed during runtime. In C, this is managed via the `malloc` function, which identifies a free block of memory and returns its starting address. In C++, this is managed via the `new` operator, which identifies a free block of memory and returns its starting address. Since this is just a pointer to some memory in the heap, there are no memory safety checks that prevents different forms of memory corruptions.

In other words, there is nothing that prevents a write operation in one heap object from overflowing into another object. This potentially changes values it should not touch. In a security context, this is often called Heap Smashing or Heap Buffer Overflow.

When an attacker overflows a buffer, they can overwrite other buffers that are sitting next to it.

They can also overwrite the internal pointers that the heap manager uses to track memory. Attackers can then use this to their advantage: by purposefully overflowing a buffer, they replace the computer's memory instructions with their own fake directions. When the program tries to manage that memory later, it gets confused and follows the attacker's fake path. This allows the attacker to rewrite critical parts of the system and force the program to execute malicious code (this is known as arbitrary code execution).

2 Design Choice

The current design has 2 main programs, the monitor process and the main process. The monitor process is entirely non-reliant on the main process and its purpose is to continuously check the main process' canaries (See "Canaries"). Canaries allow more flexibility and scale, compared to guard pages which was the initial concept to use. Guard pages would likely be limited to the amount of PMP regions which are too few for our purpose, although more security guarantees could be granted with hardware interrupts instead of pooling with canaries. Along with canaries, the canary metadata is stored in the main process which

is write protected by a PMP region. Our way of handling errors/malicious behavior is described in "Important details".

The main process would be a general code file where code, specifically `malloc`, is used. `malloc` was designed to be a simple, but the underlying code includes new canary logic in the heap. The heap was designed tightly with `malloc`, and as such was also how allocated `malloc` objects are placed in memory (See "Heap").

2.1 Canaries

We implemented our security canaries by appending an 8-byte canary to the end of every memory block allocated within the process' heap. When a process invokes `malloc`, our allocator reserves the requested space plus an additional 8 bytes. The canary is then placed at the tail end of this allocated object, acting as an indicator to detect buffer overflows.

To maintain a secure reference for validation, we store canary information in a dedicated memory region titled the *Canary Metadata*. This region is 4096 bytes in size and made up of one giant `CanaryTable` structure that only contains `CanaryObjects`. It has enough capacity to track 256 `CanaryObjects`.

A `CanaryObjects` is 16 bytes and consists of:

1. Canary Value (8 bytes), The unique random value generated for that specific allocation.
2. Pointer (8 bytes): A reference to the specific memory address where the canary resides in the heap

2.2 Heap

The current heap design is dependent on two regions of memory. The first area, the heap metadata section, stores all the important information about the heap objects, and is in short a doubly linked list linking the areas of memory together using a forward and backward pointer, with an extra bool containing if an object is in use. The second area of memory is the actual heap memory where the pointers from the metadata section are pointing, and is the area where you read and write. By keeping the metadata separated from the main heap area of memory, we increase the complexity of performing a normal heap overflow from the heap into the metadata.

To increase performance, we initialize the metadata section by dividing the memory of the heap region into equally sized smaller chunks, called blocks. We then map each block to a slot in the metadata section and link them using the doubly linked list. This in itself creates a fragment free list of the whole memory and allows us to allocate memory efficiently and quick.

When allocating memory there are some security and anti-fragmentation techniques worth considering. The first security technique is randomization, making prediction of which region of memory is to be allocated more difficult. By introducing randomization, by randomizing which block of memory is given to the user, we make it more difficult for an adversary to predict the layout of the heap, potentially facilitating arbitrary code execution.

The second technique addresses memory fragmentation. Initially, the heap is partitioned into equally sized blocks. If we allocated these blocks as is, it would lead to fragmentation and cap memory size allocation max to the size of a single block. To address these concerns, we implemented a trimming and extend function to dynamically merge blocks into a single

larger one, or dynamically trim blocks into smaller sizes of themselves. This method makes it possible for us to reduce fragmentation, while at the same time maintain high performance in the case that a allocation request matches a block size to some degree.

The third technique is utilized to prevent use after free vulnerabilities. On freeing a section of memory, which de-allocates the memory and frees up the block in the metadata, we zero the entire block's memory region. On allocation, the memory of the block is verified that it is zeroed, and in the case that it is not, the program is suspended.

2.3 Canary Trap

A custom trap handler is used to determine if write operations to the canary metadata section are allowed or not. Instead of temporarily unloading the PMP section in the functions that want to modify this section (to add and remove canaries), we let the write operation throw an exception, then verify that everything is correct in the trap handler before performing the attempted operation. To perform the operation that caused the exception, the original instruction that failed is parsed and executed with values from the original registers.

3 Important Details

Currently when there are errors/malicious behaviour occur, we handle them by running the process indefinitely without doing anything with `while(1){}`. We could perhaps have had a better error handler that would quit the program with an error code, but designing and developing an appropriate error handler was not done in time and therefore settled with a defined behavior we could easily troubleshoot.

3.1 Canaries

All important metadata for canaries are stored after the text segment (although heap metadata is in-between). The idea was that since the text-segment is read-only, we wanted to include similar areas that will be read-only aswell.

One problem about our canaries is how the amount of canary is calculated. Currently, it is statically declared through `#define` declarations. Which is fine, but this would make it difficult to scale better in the future. A bigger problem is race conditions between `app0`'s `check_canary` function and when `app1` adds a canary to it's canary table. Since `app0` continuously checks `app1`'s canary table for entries, there was a possibility that the check function would trigger before we had written the canary to the selected location on the heap. The solution was to write the canary to the heap first, then add it internally to the canary table. However, there may be bugs that still causes this race condition.

We also use address randomization to introduce unpredictable patterns for canary metadata positioning. Randomization is based on a PRNG that seeds `s3k_get_time()` function.

3.2 Heap

While randomizing values (for canaries) and locations in the heap is an important security feature for preventing and increasing the complexity of performing a multitude of attacks the random implementation currently in place is very predictable and lacks entropy. The randomization is a simple PRNG and uses the system time xored with `0x55555555` to get

a set seed. After the first seed has been generated, it generates the next number using the previous generated number, again, not very random.

Another important point to mention is that the more full the heap is, the less random the positioning of new heap objects are. If there is only a single block to select from, the random number will not matter, it will select that one (given that it fits). Currently, there is also nothing protecting the heap metadata specifically. The region is not directly protected by PMP and can therefore be modified.

3.3 Canary Trap

The canary trap has to verify that everything is happening in the correct way, to prevent an adversary from abusing the code and unlocking the PMP section for an extended period of time, allowing them to remove all canaries. There are a few checks used in the trap handler to avoid it being misused by an adversary. First of all, the trap handler uses a separate stack, which means that it will not be affected by previous stack corruptions, instead we can just verify that the stack pointer is pointing to this separate stack. Furthermore, the EPC register is compared to zero, and if it is zero, the program terminates since that would mean that the trap was called directly and not caused by an exception. Finally, the EPC is set to the value 1 to prove to future function calls that we are currently inside the trap, thereby allowing us to verify the execution path.

Since all write operations to the metadata will fail, the trap handler must perform multiple actions:

1. Verify that the stack pointer is correct and set $EPC = 1$
2. That the exception is a write operation to the canary metadata section
3. That write operation is allowed from the location that caused the exception.
4. Temporarily unload PMP region, and execute the instruction that was attempted.
5. Return to the next instruction (after the one that caused an exception)

To be able to define a limited set of functions that are allowed to write to the metadata section, a separate subsection in the text segment called *.text.critical_func* was created. Only two functions are placed in this section, the one that adds a single canary, and the one that removes one canary. The trap handler can then compare the exception address to determine if it points to this section or not.

4 Why it Works

4.1 Preventing buffer overflows

Canaries do not prevent buffer overflows completely, but can detect and react to them. The goal of this implementation is therefore not to create a theoretically perfect solution to prevent all buffer overflows, but to make it as difficult as possible to use them maliciously.

In our implementation, canaries are generated as random 64-bit values and placed in the end of heap objects. If a write operation in the heap overflows from one allocated block, it will overwrite the canary value, and unless it happens to write the same value as the randomly generated one, the monitoring process will be able to tell that an overflow has occurred. Since the monitor is using polling, it is possible for one object on the heap to have an overflow and then be freed before the next poll occurs, avoiding detection. To prevent this, canaries are also verified before they are removed.

One attack that our solution cannot prevent is if an adversary is able to a buffer overflow to gain RCE, then read the expected value for the overwritten canary and overwrite it with the expected value all before the next poll occurs. Although this is a possibility, such an attack would require substantial effort compared to if our solution had not been used at all.

4.2 Preventing attacks against canary monitoring

Since the canaries are added and handled on the same process that we try to protect, there are many ways an adversary could try to modify memory to break the canary monitoring system. Our solution prevents unintended writes to the canary metadata section by employing a read only PMP section and a simple type of control flow verification.

If an adversary tries to write directly to the metadata section at any point, by for example overwriting a pointer so that it points to this section of memory, the program will throw an exception (since the section is read-only) and the trap handler will determine the write to be illegal since it was not attempted from a function part of the *.text.critical_func* section. It would also be difficult for an adversary to unload the PMP section by calling our unload function since it locks the program if $EPC \neq 1$, which it is unless the code is currently in the trap handler. An adversary could still theoretically use return oriented programming to directly call the internal s3k function *s3k_pmp_unload* if they are able to guess and set the arguments correctly.

Since the *remove_canary* function has to be able to remove canaries, there is no way to prevent an attacker from calling this function using return oriented programming. Instead of making it impossible to call, the canary metadata section has been designed around placing canaries at random locations, making it difficult to predict where any single canary is stored.

Since the canary monitoring itself happens in a separate, trusted, process, there is no simple way for an attacker to affect the the monitoring code. Furthermore, the *.text* section is made non-writable at program start to avoid code modifications.

Contribution - Måns

My main contribution to the project was the safety measures to protect the canary metadata section. This involves setting up and managing capabilities for the process, and handling of loading and unloading of the PMP capability. One part of this was to create the trap handler that intercepts write attempts to the canary metadata and only run them if some criteria are met. Writes are only allowed if the stack pointer is correct, and the instruction was attempted from an allowed function stored in the *.text.critical_func* section. Another part of the trap handler that I implemented was instruction parsing and register restoration so that the original write instruction could be performed inside of the trap handler. I have previously created a RISC-V emulator, so I was already somewhat familiar with the instruction types, making this a good task for me.

I also contributed to the malloc implementation by designing the data structure and by reducing the fragmentation from an unusable state, to something much better. This involved creating some fragmentation tests, and modifying the code so that blocks were combined recursively, and not just once.

When it comes to canaries I was responsible for fixing a few bugs, mostly related to race conditions. One important feature that I implemented is the verification of canaries before they are removed. Without this, buffer overflows would not be detected if the heap object was freed quickly after. Another race condition that I solved was in the canary monitoring function. The problem is that if there is a context switch during the canary monitoring, the first part of the function will use outdated data, leading to some false positive detections. To prevent this, the canary metadata is checked a second time, after it has been detected as an overflow, to see if the canary metadata has been removed. This is in no way a perfect solution, but allows the program to work at all. This type of race-condition was not detected before we added interrupts to write operations to the metadata section, so we did not have much time to fix it.

I was also responsible for making the text section RX, and not writable, along with discussing how other parts of the program could be implemented.

Contribution - Emil

During the initial weeks I actively participated in designing the implementation of the heap and malloc. The initial design was done in collaboration between me and Måns. More specifically, I designed the initial implementation of the trim and extend function, aimed at reducing the fragmentation in malloc. I also created the simple randomize method, and was responsible for introducing randomization in the various parts of the secure heap such as the block by block randomization and block traversal (based on randomization). In addition, I also designed the free function in malloc, and introduced protection against use after free by zeroing allocated memory on free, and performing a memory integrity check during malloc. Additionally, I made it so that the monitor process will suspend the guarded process on canary check fail.

As for bugs, I managed to identify and fix a race condition bug related to the trap handler, which involved reading before writing.

Contribution - Lukas

During this project I worked particularly with canaries and s3k-specific configuration such as using the s3k-capability functions to initiate the two apps; app0 and app1. I, together with Samuel, handled how capabilities were derived from the initial capabilities such as memory, time and PMP devices, all done in app0's `main.c`. Some particular information was needed e.g. the RISC-V-specific information that Måns provided helped to understand that PMP regions have priority and that PMP modules have to be loaded in certain indexes for our purpose of toggling read-only PMPs.

It is also worth to note that both me and Samuel worked together here. We both had to understand the syntax for the `derive` function, then we could continue with setting up all the other capabilities. Collaboratively, we worked with memory capabilities. On my own I made sure time and PMP capabilities were setup. I also tried to distribute `monitoring` capabilities during an early design, but ran into some issues.

Together with Samuel during pair-programming sessions were interfaces for handling canaries inside `canary.c` and `canary.h` created. Due to pair programming, what I contributed is also influenced, but I did the `remove_canary` function, for example. We also discussed how canaries should be designed, e.g. which type should be used for `canary` and its associated `heap_canary_pointer` in the `CanaryObject`, although it got somewhat changed towards the end of the project.

To summarize:

- setup of s3k-capabilities in app0
- Canary logic and `remove_canary` function

Contribution - Samuel

Most of my work was on creating the the Canary code and canary table, and it was mostly done with pair programming, so it's difficult to pinpoint code that no one but me has touched. However, I did provide more contribution on certain parts. Such as deciding the structure for our canary table and canaries.

I created

- the `init_canary_table` function (which got updated with what values to define it as empty, but the structure stayed the same).
- the `add_canary` function. However, i did not add the random function to it, as that was done in later stage by someone else.
- the `check_canary` function.

As well as I did have my hands in bits of everything in the `canary.c` and `canary.h` file.

I also created the `utils` file with the purpose of having a centralized file with all our macros and needed functions. However, it uses multiple defines and functions from the `s3k utils` file. As well as my teammates have added more defines to it themselves.

I set up a lot of the capabilities inside the `main.c` file inside `app0` folder. However, I did this with Lukas, and the file has been repetitively updated by everyone.